

domain-architecture-skills：让 AI 先建模、再选架构、最后写代码

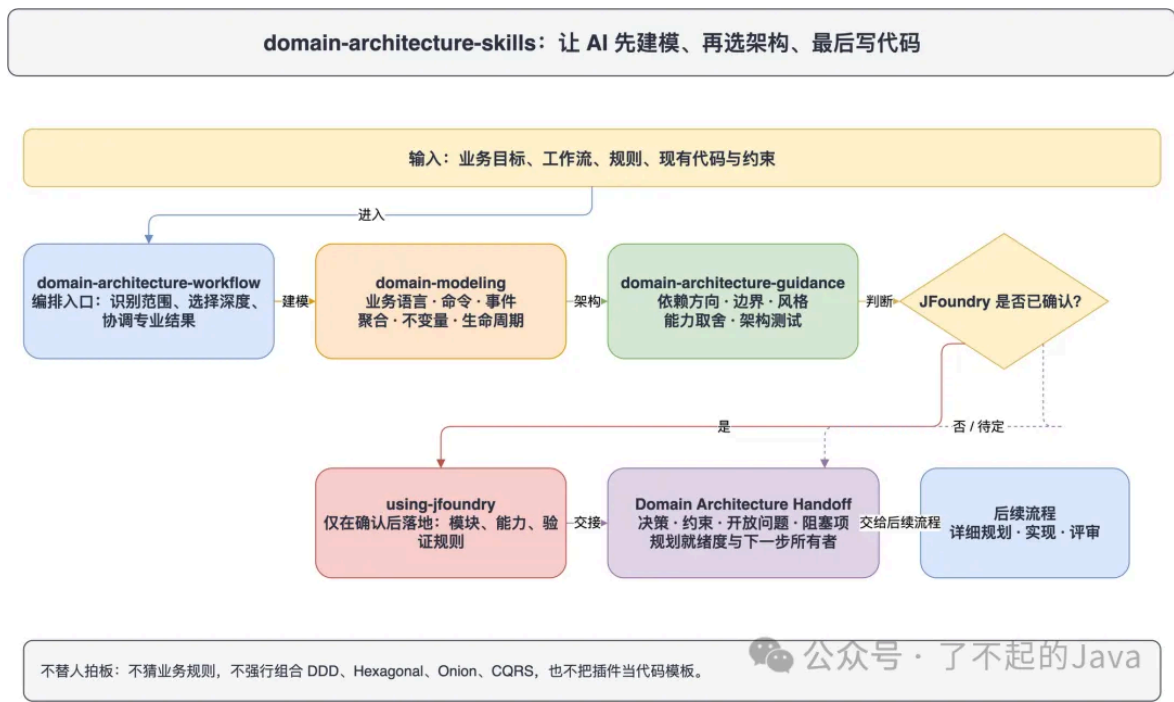
了不起的Java 2026年9月4日 08:30 湖北

domain-architecture-skills：让 AI 先建模、再选架构、最后写代码

上一篇我们讨论了 AI 为什么会写出“结构正确、业务错误”的代码。

问题通常不在生成能力，而在工作顺序：需求里缺少的业务事实，被 AI 当成了默认规则；尚未做出的架构选择，被常见模板替我们做了；最后，代码只是把这些未经确认的假设实现得很完整。

domain-architecture-skills 的定位，就是把这段设计期工作拆开。它不是一个 DDD 代码生成器，也不是把所有项目都改造成 Hexagonal、Onion 或 CQRS 的模板，而是一组让 AI 按责任边界协作的专业入口。



请使用 DDD、Spring Boot 和 JFoundry, 实现订单取消接口。

它很可能马上生成 Controller、ApplicationService、Aggregate、Repository 和测试。但这段提示没有说明:

- 订单取消的业务边界是什么;
- 哪些状态允许取消, 哪些状态必须拒绝;
- 取消是否需要退款或通知其他系统;
- 聚合边界是否已经确认;
- 项目是否真的需要 CQRS、Outbox 或分布式锁。

AI 当然可以继续写, 但它写的是一份推测。插件的第一条原则, 是先判断哪些内容已经有证据, 哪些内容仍然需要业务确认。

2. 四个入口, 各自负责什么?

插件包含四个入口。它们不是四个可以随意混用的提示词, 而是四种不同的责任。

domain-architecture-workflow: 端到端编排入口

当任务要从需求一路走到架构决策和实施交接时, 从这个入口开始。它负责识别业务目标、参与者、工作流、约束和不确定性, 再决定是否需要领域建模、架构指导以及可选的 JFoundry 落地。

它是协调者, 不会把各个 specialist 的结果重写成一套自己的模板。每个阶段都有独立状态, 遇到阻塞时只暂停依赖该事实的后续工作。

domain-modeling: 先回答“业务上是什么”

它从业务工作流、需求、现有 API、表结构和代码证据出发, 提取:

- 统一语言和术语含义;
- 命令、事件、状态和例外;
- 业务规则与不变量;
- 聚合、实体和值对象候选;
- 外部事实、应用协作和读取需求。

它不会因为看到几张表就自动创建几个聚合, 也不会业务含义不明确时替人确认规则。结果中会区分 `confirmed`、`inferred` 和 `proposed`, 让暂时的工作假设保持可见。

domain-architecture-guidance: 再回答“边界如何保护业务”

领域模型明确之后，才讨论依赖方向、分层、Onion、Hexagonal、CQRS、Repository 和架构测试等问题。

这个入口的职责是把已确认的业务决策映射成架构约束。例如：某个不变量需要由聚合保护，应用层负责用例编排，持久化细节留在适配器；但它不会根据包名或框架能力反推业务边界。

如果架构选择会改变聚合所有权、上下文含义或规则归属，就应该回到领域建模，而不是让架构建议悄悄改写业务模型。

using-jfoundry：最后才做框架落地

只有在项目已经确认使用 JFoundry，或者明确要求 JFoundry 实现指导时，才进入这个入口。

它负责把已选架构落到 JFoundry 的模块、依赖、包角色、聚合持久化、可靠消息、运行时装配和 ArchUnit 验证。它不会因为 JFoundry 提供了某个能力，就替项目选择 CQRS、Outbox、锁或某种运行时。

如果 JFoundry 仍然是“未决定”，前面的框架中立建模和架构分析仍然可以继续；不应该为了等待一个框架选择而阻塞业务分析。

3. 一次真实的“订单取消”协作

下面是一份更适合交给插件的输入：



请使用 `$domain-architecture-workflow` 分析订单取消用例。

业务目标：允许用户取消尚未发货的订单。

已有证据：订单状态定义、现有取消接口、客服操作说明。

已确认规则：已发货订单不能取消。

待确认问题：支付后是否退款；重复取消是否幂等；取消事件的接收方。

技术约束：Java 25；当前项目使用 JFoundry；运行时为 Spring Boot。

希望的下一步：输出领域模型、架构指导和可实施的交接结果，先不要改代码。

在这个过程中，插件应该先保留“退款规则”和“重复取消语义”这两个开放问题。它可以继续分析不受影响的内容，例如“已发货不可取消”这一已确认规则可能需要被订单聚合保护；但它不应该自行决定支付后的处理方式。

不依赖这些开放问题的架构分析可以继续，依赖退款和幂等语义的决策则应暂停。业务团队补充证据后，再刷新受影响的领域建模和架构指导。确认 JFoundry 后，`using-jfoundry` 才负责选择对应的模块和适配器。

4. 为什么需要 Domain Architecture Handoff?

专业分析和写代码之间经常有一段断层。AI 换了一个会话，或者开发者直接拿着一段总结开始实现，最容易丢掉的恰恰是“哪些是事实，哪些是猜测”。

Domain Architecture Handoff 是插件的交接产物，用来保留：

- 领域建模和架构指导各自的状态；
- 已确认的决策及其证据；
- 约束、假设、开放问题和阻塞项；
- 当前模型与目标意图之间的差异；
- JFoundry 是否适用，以及框架落地的范围；
- 最小的规划就绪增量和下一步所有者。

它不是详细实施计划，也不是自动生成的任务清单。它的作用是让后续规划、实现或评审接住专业结果，而不是重新猜一遍前面的业务含义。

5. “未决定”是一个有效状态

许多 AI 工作流的问题在于，只有“是”和“否”，没有“还不知道”。

在插件里，下面这些状态都可以被保留：



JFoundry: 未决定

CQRS: 未决定，当前读写需求尚不足以推动选择

退款规则: 需要业务确认

聚合边界: 已有候选，但尚未被业务负责人接受

这不是流程失败，而是对不确定性的准确描述。只有当某个未决问题确实阻塞下一步时，阶段才需要返回 `needs-input`；不相关的未来问题可以保留为开放项，不必让整个任务停下来。

6. 它和通用编码流程如何配合？

插件负责领域和架构专业结果，通用流程负责规格、计划、任务、代码、测试和评审。两者的边界应该保持清楚：



需求与现有证据

-> domain-architecture-workflow

```
-> Domain Modeling Result
-> Architecture Guidance Result
-> (确认 JFoundry 后) JFoundry Implementation Guidance Result
-> Domain Architecture Handoff
-> 详细规划、实现与评审
```

例如，后续规划流程可以根据交接结果拆出“先实现订单聚合规则，再实现应用用例，最后接入持久化适配器”的任务，但不应该把交接中的 `proposed` 直接当成已确认事实。实现过程中如果发现业务含义变化，还要回到领域建模阶段重新确认。

7. 什么时候不应该使用完整 workflow?

插件明确不鼓励把所有项目都做成复杂 DDD。

如果功能只是简单的后台字典维护，没有复杂生命周期、跨对象不变量、外部协作或语义冲突，直接完成 CRUD、测试和发布可能更合适。此时可以不用完整的战略建模、聚合设计和框架落地链路。

值得提高分析深度的信号包括：

- 同一个术语在不同团队中含义不一致；
- 状态转换存在例外和补偿；
- 多个对象需要在同一边界内保持一致；
- 业务规则正在被 Controller、Service、DAO 分散实现；
- 查询模型、写模型和外部事件有不同的变化原因。

流程的目标不是增加文档数量，而是把会改变系统行为的决策提前暴露出来。

8. 如何在本地使用这个插件？

通过 Git 来源安装到 Codex：

```
codex plugin marketplace add xfoundries/domain-architecture-skills
codex plugin add domain-architecture@xfoundries
```

如果要从当前检出目录做本地开发或验证，则在插件目录中执行：

```
codex plugin marketplace add .
codex plugin add domain-architecture@xfoundries
```

`xfoundries marketplace` 名称一次只能对应一个来源；在 Git 来源和本地检出之间切换前，需要先移除已有 `marketplace`。修改插件内容后，也需要在目标智能体中重新安装或更新，以刷新缓存。

安装完成后，最重要的不是记住四条命令，而是从正确入口开始：端到端任务用 `domain-architecture-workflow`，只需要建模时用 `domain-modeling`，只做边界评审时用 `domain-architecture-guidance`，确认 JFoundry 落地时再用 `using-jfoundry`。

结尾

`domain-architecture-skills` 的价值，不是让 AI 生成更多类，而是让 AI 在生成代码之前先回答三个问题：

1. 业务事实是什么，哪些地方还不确定？
2. 已确认的业务决策需要什么边界来保护？
3. 什么时候才应该把这些决策落到具体框架和代码？

当这三个问题被分开处理，AI 的角色就从“根据关键词套模板”变成了“基于证据完成分析、映射和实现”。业务专家仍然负责业务含义，项目团队仍然负责架构取舍，AI 则负责把结果稳定地传递到代码和验证中。

下一篇我们继续讨论：**从 Domain Architecture Handoff 到实现，AI 协作中的责任边界如何划分？**

相关资料

- JFoundry: <https://github.com/xfoundries/jfoundry>^[1]
- 领域架构插件: <https://github.com/xfoundries/domain-architecture-skills>^[2]

JFoundry · 目录

上一篇

AI Agent 为什么会写出“结构正确、业务错误”的代码

下一篇

从 Domain Architecture Handoff 到实现：AI 协作中的责任边界

